

Semester: CE-5

Comsats University Islamabad Attock Campus



Submitted By: Sumaira Safeer
Registration number: FA22-BCE-019
Submitted To: Mr. Qazi Zia
Course Name: Database Syatem
Program: BS Computer Engineering
Topic: OpenEnded Lab Report

Department of Computer Engineering

Library Management System

Database Integration with a Web-Based Application

Design and connect a Database with a Webpage (e.g. using PHP)

Design a database of your own choice (you are open to use any DBMS software). Design a webpage (using any tools of your choice) and connect your designed database to your webpage from where you can edit your database.

Lab Report:

1. Explain choosing the software tools you use in the conducting your open ended lab.
2. Explain the issues you faced in the design and connecting your database to the webpage.
3. Explain the overall procedure you followed for completing the lab.
4. Give alternate tools that can be used to conduct the given open ended lab.

Part 1. Software Tools Selection

Selected Tools (Xampp Server, MySQL, PHP):

- Database Management System: MySQL 8.0
- Chosen for its robust reliability, widespread community support, excellent integration with PHP.
- Free and open-source.
- Extensive documentation and learning resources available.

Web Server: Apache 2.4

- Industry standard for PHP applications.
- Easy to set up using XAMPP package.
- Built-in support for MySQL and PHP

Server-Side Language: PHP 8.0

- Native support for MySQL databases.
- Simple syntax for beginners.
- Large ecosystem of libraries and frameworks.

Frontend Technologies:

- HTML/CSS for frontend structure and styling.
- Bootstrap 5 for responsive design.
- JavaScript for client-side validation

Development Environment:

- XAMPP 8.0 as the local development stack.
- Includes Apache, MySQL, PHP, and phpMyAdmin.
- Cross-platform compatibility.
- One-click installation and configuration.

Part 2. Design and Connection Issues

Database Design Challenges:

1. Schema Design:

- Initially struggled with normalization decisions.
- Resolved by creating separate tables for authors and categories.
- Implemented foreign key constraints for data integrity.

2. Character Encoding:

- Encountered UTF-8 encoding issues with special characters.
- Fixed by setting proper collation in MySQL (utf8mb4_unicode_ci).

Connection Issues:

1. Database Connection Security:

- Initial exposure of database credentials in PHP files.
- Solution: Created a separate configuration file and added it to .gitignore.
- Implemented PDO with prepared statements for SQL injection prevention.

2. CORS and Access Control:

- Cross-origin resource sharing issues during development.
- Resolved by proper configuration of Apache headers.

Frontend Integration Challenges:

1. Form Validation:

- Dual validation implementation (client and server-side).
- Created reusable validation functions.

2. Asynchronous Updates:

- AJAX implementation for real-time updates.
- Error handling for failed database operations.

Part 3. Implementation Procedure:

Name	Date modified	Type	Size
 add_book	12/23/2024 7:14 PM	HTML Document	2 KB
 add_book	12/23/2024 6:39 PM	PHP File	1 KB
 database	12/24/2024 6:50 PM	SQL Text File	1 KB
 db_connection	12/24/2024 6:41 PM	PHP File	1 KB
 images1	12/23/2024 7:03 PM	JPG File	6 KB
 index	12/23/2024 7:15 PM	HTML Document	1 KB
 styles	12/23/2024 7:21 PM	CSS Source File	3 KB
 view_books	12/23/2024 7:15 PM	PHP File	2 KB

Index HTML:

```
</!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Library Management System</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body >
  <header>
    <h1>Library Management System</h1>
    <nav>
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="add_book.html">Add Book</a></li>
        <li><a href="view_books.php">View Books</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <h1>Welcome to the Library</h1>
    <p>Manage your library with ease!</p>
  </main>
  <!-- Footer Section -->
  <footer>
    <p>&copy; 2024 Library Management System. All rights reserved.</p>
```

```
</footer>
</body>
</html>
```

Database Creation:

```
-- Create the database
CREATE DATABASE IF NOT EXISTS library_management;
USE library_management;

-- Create the books table
CREATE TABLE IF NOT EXISTS books (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    genre VARCHAR(100),
    status ENUM('available', 'borrowed') DEFAULT 'available',
    added_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create the users table
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(255) NOT NULL,
    password VARCHAR(255) NOT NULL,
    role ENUM('admin', 'member') DEFAULT 'member',
    registered_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

PHP Script:

```
<?php
include 'db_connection.php';

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $title = $_POST['title'];
    $author = $_POST['author'];
```

```

$genre = $_POST['genre'];
$sql = "INSERT INTO books (title, author, genre) VALUES ('$title', '$author', '$genre')";
if (mysqli_query($conn, $sql)) {
    echo "New record created successfully";
    header("Location: view_books.php"); // Redirect to view books page
} else {
    echo "Error: " . $sql . "<br>" . mysqli_error($conn); }
mysqli_close($conn);
}
?>

```

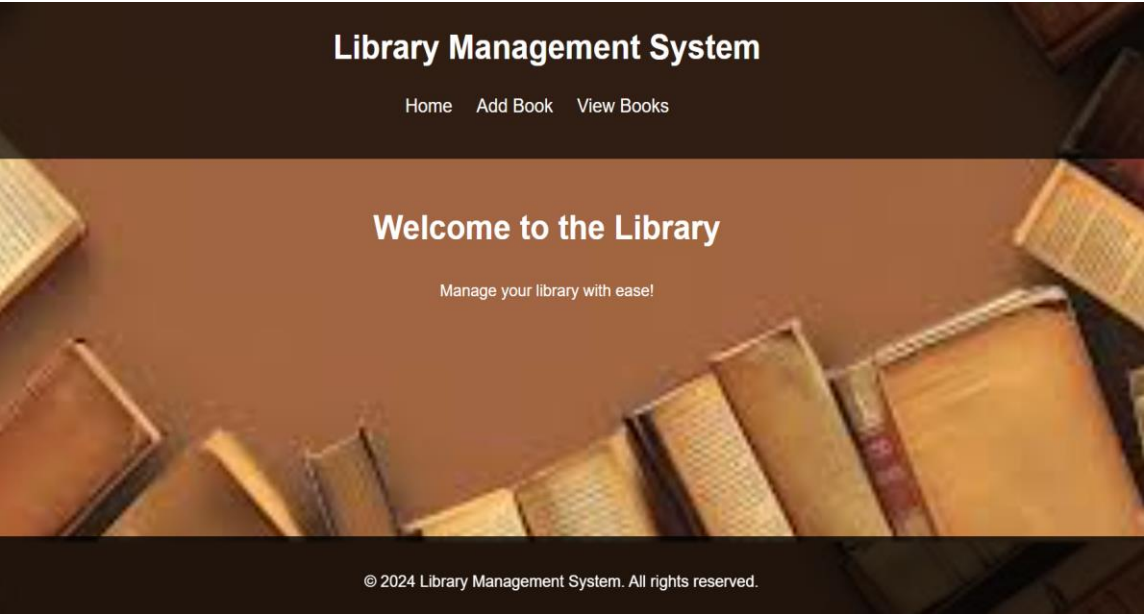
db_connection:

```

<?php
$server = "localhost";
$username = "root"; // replace with your MySQL username if different
$password = ""; // replace with your MySQL password if set
$database = "library_management"; // Update this to match your actual database name
// Create connection
$conn = mysqli_connect($server, $username, $password, $database);
// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";
?>

```

Result:



4. Alternative Tools

Alternative Database Systems:

1. PostgreSQL:

- More feature-rich than MySQL.
- Better support for complex queries.
- Stricter SQL compliance.

2. SQLite:

- Lighter weight, file-based database.
- Good for smaller applications.
- No separate server required.

Alternative Web Servers:

1. Nginx:

- Better performance for static content.
- Lower memory footprint.
- Growing popularity in production environments.

Alternative Backend Languages:

1. Node.js with Express:

- JavaScript throughout the stack.
- Large package ecosystem.
- Event-driven architecture.

2. Python with Django/Flask:

- Robust ORM.
- Extensive library support.
- Clean syntax.

Alternative Frontend Frameworks:

1. React.js

- Component-based architecture.
- Virtual DOM for better performance.
- Large ecosystem.

2. Vue.js

- Gentle learning curve.
- Progressive framework.

- Good documentation.

Conclusion:

This implementation demonstrates a basic but functional database-driven web application. The chosen tools (MySQL, PHP, Apache) provide a solid foundation for learning database-webpage integration while offering room for scaling and improvement. The alternative tools section shows there are many valid approaches to solving the same problem, each with its own advantages depending on specific requirements.

The project successfully implements CRUD operations, demonstrates proper security practices, and provides a responsive user interface. Future improvements could include implementing user authentication, adding more complex search functionality, and expanding the database schema to handle more sophisticated data relationships.